Prob 1. (10 pts) We learned how to implement Q31 multiplication using assembly, with the code shown below:

```
@ int32_t mp_mult_q31_s(int A, int B)
mp_mult_q31_s:
    smull   r1, r0, r0, r1
    lsl     r0, #1
    add     r0, r0, r1, lsr #31
    bx      lr
```

This code was based on the following analysis. First, after the multiplication, we have the following in R0 and R1:

$$\overbrace{\boxed{b_{63} \mid \quad \ldots \quad \mid b_{33} \mid b_{32}}}^{\text{HW = R0}}\overbrace{\boxed{b_{31} \mid \quad \ldots \quad \mid b_{1} \mid b_{0}}}^{\text{LW = R1}}$$

Then, the next two lines of code shifts the above number to the right by 31 bits, leading the value of R0 being the required return:

$$\boxed{b_{62} \mid \quad \ldots \quad \mid b_{32} \mid b_{31}}$$

Now, consider the Q15.16 multiplication. By comparing Q31 and Q15.16, we know that we can implement Q15.16 multiplication by modifying the Q31 multiplication code with appropriate shift to return

$$\boxed{b_{47} \mid \quad \ldots \quad \mid b_{17} \mid b_{16}}$$

Now, modify this function to perform Q15.16 multiplication. Make all necessary changes, including the prototype in the comment. Use function name `mp_mult_q15p16_s`

Soln:

```
@ int mp_mult_q15p16_s(int A, int B) or
mp_mult_q15p16_s:
    smull   r1, r0, r0, r1
    lsl     r0, #16
    add     r0, r0, r1, lsr #16
    bx      lr
```

> -:-

Remarks of the solution:

- It is easier to figure out how many bits to shift R1 to the right. This is 16, as the data format is Q15.16.
- Now, we can figure out how many bits we need to shift to R0 to the left. b_{47} of old R0 should be the MSB of R0 after the shift, so that the left shift should be 47-31 = 16.

Note that if we use Q16.15, then we should have

```
mp_mult_q16p15_s:
    smull   r1, r0, r0, r1
    lsl     r0, #17
    add     r0, r0, r1, lsr #15
    bx      lr
```

> -:-

Prob 2. (15 pts, 5 pts each) Consider the following code for a 32-bit system:

```
ldr     r0, =0b1101_1101  @ just assume it works
lsls    r1, r0, #1
lsrs    r2, r0, #3
```

Determine the values of R1 and R2 in **hexadecimal** after running the above code. Also determine the value of the C flag after running the above code assuming the system follows the working principle of the ARM processor we discussed in the class.

Solution.

```
R1 = 0b1_1011_1010 = 0x1BA
R2 = 0b0001_1011 = 0x1B
C flag = 1
```

> -:-

Remarks of the solution:

- The shift operations are easy to see.
- To see the change of the C flag, we need to see what is the bit last shifted out. It was b_2 of R0, which is 1.

> -:-

Prob 3. (30 pts total, 5 pts each) We have learned in the class that using the barrel shifter, we can scale a number for some special cases with addition, subtraction, and reversed subtraction instructions as well as shift instructions. We have also practiced some calculations in Expls 1 and 2 in Lctr 16. Assume **signed** integers `A` and `B` are saved in R0 and R1, respectively, and the numbers are small enough that no *overflow* will occur. Write a **single-line** assembly code based on Op2 to perform the following needed scaling.

```
(a) A = 16 * B
(b) A = 17 * B
(c) A = 15 * B
(d) A = B / 16
(e) A = 17 * B / 16
(f) A = 15 * B / 16
```

Soln:

```
(a) A = 16 * B:         lsl  r0, r1, #4
(b) A = 17 * B:         add  r0, r1, r1, lsl #4
(c) A = 15 * B:         rsb  r0, r1, r1, lsl #4
(d) A = B / 16:         asr  r0, r1, #4
(e) A = 17 * B / 16:    add  r0, r1, r1, asr #4
(f) A = 15 * B / 16:    sub  r0, r1, r1, asr #4
```

> -:-

Remarks of the solution:

We have to decompose those numbers that cannot be expressed into 2^n (n can be negative) into the arithmetic operation between 1 and 2^n. For example:

- $17 = 1 + 2^4$
- $15 = 2^4 - 1$
- $17/16 = 1 + 2^{-4}$
- $15/16 = 1 - 2^{-4}$

> -:-

Prob 4. (10 pts, 5 each) Assume we have the following assembly function

```
@ uint16_t mp_func_s(uint16_t a);
mp_func_s:
    ldr    r1, =15
    bic    r0, r0, r1, lsl #4
    ldr    r2, =10
    orr    r0, r0, r2, lsl #4
    bx     lr
```

1. If a = 0x$H_3H_2H_1H_0$, what is the value of `b` after running `uint16_t b = mp_func_s(a)`?
2. Translate the above assembly function to a C function with the following prototype: `uint16_t mp_func_c(uint16_t a)`.

Soln:

Part 1: a = 0x$H_3H_2AH_0$.

Part 2:

```
uint16_t mp_func_c(uint16_t a) {
    a &= ~(15 << 4);
    a |= (10 << 4);
    return a;
}
```

> -:-

Remarks of the solution:

- Part 2 is based on Part 1.
- To finish Part 1, we just need to perform each step carefully. `ldr r1, =15` and `bic r0, r0, r1, lsl #4` will zero the second hexadecimal digit from right. The next two lines will *assign* the value.

> -:-

Prob 5. (16 pts total, 4 pts each) Assume `A` and `mask` are `uint8_t` variable saved in R0 and and R1, respectively. Assume their values are 0xAD and 0x03, respectively. Determine the values of the least significant bytes of R2 to R5 in **hexadecimal** after running the following assembly code

```
    lsr   r2, r1, #2
    lsl   r2, #2
    bic   r3, r0, r1
    orr   r4, r3, r1
    eor   r5, r0, r1
```

Soln:

```
Given conditions:

        r0 = 0xAD = 0b1010_1101
        r1 = 0x03 = 0b0000_0011

Solutions:

        r2 = 0x00
        r3 = 0b1010_1100 = 0xAC
        r4 = 0b1010_1111 = 0xAF
        r5 = 0b1010_1110 = 0xAE
```

> -:-

Remarks of the solution:

- It is easy to do this problem once we write the number in binary format.

> -:-

Prob 6. (30 pts total, 4 pts for each C fn and 6 pts for each ASM fn) Assume `A` is a `uint16_t` variable, written as 0x$H_3H_2H_1H_0$, intended to express an arbitrary number in hexadecimal format. Here H_k is the kth hexadecimal digit written in the normal way where the right-most digit is the least significant digit. Write **C and ASM** functions to perform the following operations to change the digit (4 bits) of interest *systematically* without affecting the other digits (bits) or shift the number:

(a). A = 0x$H_3H_2H_1H_0$ => A = 0x$H_3H_2 5H_0$

(b). A = 0x$H_3H_2H_1H_0$ => A = 0x$H_2H_1H_0 3$

(c). A = 0x$H_3H_2H_1H_0$ => A = 0x$7H_3H_2H_1$

Note that for the assembly implementation, here are the requirements:

- Write each of the solution in a function. For example, for Part (a), the prototypes of the C and ASM functions can be `uint16_t mp_func_a_in_c(uint16_t A)` and `uint16_t mp_func_a_in_s(uint16_t A)`, respectively.
- For the C functions, just write the definition; for the ASM function, write each function's prototype clearly.
- You can use any registers without having to push them to the stack.
- If you need to load a value to a register, use the format of `LDR Rd, =num`.

Soln

C implementations:

(a)

```
uint16_t mp_func_a_in_c(uint16_t A) {
    A &= ~(15u << 4);
    A |= 5u << 4;
    return A;
}
```

(b)

```
uint16_t mp_func_b_in_c(uint16_t A) {
    A <<= 4;
    A |= 3;
    return A;
}
```

(c)

```
uint16_t mp_func_c_in_c(uint16_t A) {
    A >>= 4;
    A |= (7U << 12)
    return A;
}
```

Assembly implementations:

(a)

```
@ uint16_t mp_func_a_in_s(uint16_t A)
mp_func_a_in_s:
    ldr  r1, =15
    lsl  r1, #4   ; mask is ready
    bic  r0, r1
    ldr  r2, =5
    lsl  r2, #4
    orr  r0, r2
    bx   lr
```

(b)

```
@ uint16_t mp_func_b_in_s(uint16_t A)
mp_func_b_in_s:
    lsl  r0, #4
    orr  r0, #3
    bx   lr
```

(c)

```
@ uint16_t mp_func_c_in_s(uint16_t A)
mp_func_c_in_s:
    lsr  r0, #4
    ldr  r1, =7
    lsl  r1, #12
    orr  r0, r1
```

```
    bx   lr
```

> -:-

Remarks of the solution:

- There can be multiple different solutions. As long we do the work, it will be fine.
- Prob (a) is a standard problem to assign a bit field of a number. We have to do it in two steps: (a) zero the field first using AND and (2) *assign* the value using ORR.
- Prob (b) is a two-step operation as well. The first is to shift to left for form 0x$H_2H_1H_0$0, and the second is to use ORR to assign 3 to the least significant digit.
- Prob (c) is similar to Prob (b). We just need to be shift 7 to the left by 12 bits. this can only be done if we load 7 into a register.

> -:-